

intlang: Formal Semantics and Type System (DEPRECATED)

1 Lexical Conventions

- **Identifiers:** An identifier is a letter or underscore followed by zero or more letters, digits, or underscores. Identifiers are case-sensitive.
- **Integer literals:** $n \in \mathbb{N} \cup \{0\}$. Only non-negative integers can be written as literals. Negative integers can be constructed using subtraction, e.g., $0 - n$.
- **Comments:** Line comments begin with `--` and extend to the end of the line.
- **Whitespace:** Ignored (spaces, tabs, newlines, carriage returns).
- **Keywords:** `let`, `in`, `if`, `then`, `else`, `end`, `include`, `vec`, `vecmk`, `vecLen`, `vecget`, `vecset`.

2 Concrete Grammar (BNF)

The concrete grammar is defined in extended Backus-Naur form:

2.1 Programs and Statements

```
prog ::= stmt*  
stmt ::= include id  
       | include " id "  
       | include " path "  
       | let id = expr ;  
       | expr
```

A program consists of optional include statements.

- Zero or more named let-bindings (with semicolons) and a final expression.
- Or, One or more named let-bindings (with semicolons) and no final expression.

2.1.1 Include Statements

Include statements can reference global libraries (no Quotes) or relative file paths (with Quotes). An include gives access to the named let-bindings from the included module (which is just another .intlang file). The .intlang ending must not be written in the include statement.

```
path ::= id path  
       | / path  
       | . path  
       | - path  
       | / id
```

2.1.2 Qualified Names

A qualified name `id.id` allows referencing variables from included modules. For example, `math.mod` refers to the `mod` binding from the `math` module.

2.2 Expressions

Expressions are defined with explicit precedence levels (from lowest to highest binding strength):

2.2.1 Comparison Operators

$$\begin{aligned} \text{expr} ::= & \text{let id} = \text{expr in expr} \\ & | \text{if expr then expr else expr end} \\ & | \backslash \text{id} . \text{expr} \\ & | \text{expr}_{\text{cmp}} \end{aligned}$$

Let-in bindings, conditionals, and lambda abstractions have lowest precedence.

2.2.2 Comparison

$$\begin{aligned} \text{expr}_{\text{cmp}} ::= & \text{expr}_{\text{cmp}} == \text{expr}_{\text{add}} \\ & | \text{expr}_{\text{cmp}} < \text{expr}_{\text{add}} \\ & | \text{expr}_{\text{add}} \end{aligned}$$

Comparison operators ($==$, $<$) are left-associative.

2.2.3 Addition and Subtraction

$$\begin{aligned} \text{expr}_{\text{add}} ::= & \text{expr}_{\text{add}} + \text{expr}_{\text{mul}} \\ & | \text{expr}_{\text{add}} - \text{expr}_{\text{mul}} \\ & | \text{expr}_{\text{mul}} \end{aligned}$$

Addition and subtraction are left-associative.

2.2.4 Multiplication and Division

$$\begin{aligned} \text{expr}_{\text{mul}} ::= & \text{expr}_{\text{mul}} * \text{expr}_{\text{app}} \\ & | \text{expr}_{\text{mul}} / \text{expr}_{\text{app}} \\ & | \text{expr}_{\text{app}} \end{aligned}$$

Multiplication and division are left-associative.

2.2.5 Application

$$\begin{aligned} \text{expr}_{\text{app}} &::= \text{expr}_{\text{app}} \text{expr}_{\text{atom}} \\ &| \text{expr}_{\text{atom}} \end{aligned}$$

Application is left-associative and binds strongest: $f\ g\ h$ parses as $(f\ g)\ h$.

2.2.6 Atomic Expressions

$$\begin{aligned} \text{expr}_{\text{atom}} &::= \text{id} \\ &| \text{id} . \text{id} \\ &| n \\ &| \mathbf{vec}\ [\text{expr}_{\text{list}}] \\ &| \mathbf{vecmk}\ [\text{expr}, \text{expr}] \\ &| \mathbf{veclen}\ [\text{expr}] \\ &| \mathbf{vecget}\ [\text{expr}, \text{expr}] \\ &| \mathbf{vecset}\ [\text{expr}, \text{expr}, \text{expr}] \\ &| (\text{expr}) \end{aligned}$$

2.2.7 Expression Lists

$$\begin{aligned} \text{expr}_{\text{list}} &::= \text{expr}, \text{expr}_{\text{list}} \\ &| \text{expr} \end{aligned}$$

2.2.8 Qualified Names

A qualified name `id.id` allows referencing variables from included modules. For example, `Math.sin` refers to the `sin` binding from the `Math` module.

3 Type System

3.1 Types and Type Schemes

3.1.1 Type Syntax

$$\tau ::= \text{int} \mid \alpha \mid \tau \rightarrow \tau \mid \text{Vec}[\tau]$$

where α is a type variable

A **type scheme** or **generic type** has the form:

$$\sigma ::= \forall \alpha_1 \dots \alpha_n. \tau$$

where $\alpha_1, \dots, \alpha_n$ are quantified type variables. When $n = 0$, we write just τ instead of $\forall. \tau$.

3.1.2 Type Environment

The type environment Γ is a finite map from variable names to type schemes:

$$\Gamma \in \text{Var} \rightarrow \text{TypeScheme}$$

3.2 Type Inference Judgment

The type judgment has the form:

$$\Gamma \vdash e : \tau \mid C$$

where:

- Γ is the type environment
- e is the expression
- τ is the inferred type
- C is a set of type constraints (equalities to be unified)

3.3 Inference Rules

3.3.1 Variable

When a variable is looked up in the environment, we instantiate its type scheme to fresh type variables:

$$\frac{\Gamma(x) = \forall \alpha_1 \dots \alpha_n. \tau \quad (\text{fresh } \beta_1, \dots, \beta_n)}{\Gamma \vdash x : \tau[\beta_1/\alpha_1, \dots, \beta_n/\alpha_n] \mid \emptyset} \quad (\text{VAR})$$

3.3.2 Integer Literal

$$\overline{\Gamma \vdash n : \text{int} \mid \emptyset} \quad (\text{INT})$$

3.3.3 Lambda Abstraction

For a lambda abstraction $\lambda x.e$, we introduce a fresh type variable α for the parameter, extend the environment, and check the body:

$$\frac{\text{fresh } \alpha \quad \Gamma, x : \alpha \vdash e : \tau \mid C}{\Gamma \vdash \lambda x.e : \alpha \rightarrow \tau \mid C} \quad (\text{LAM})$$

3.3.4 Application

For an application $f x$, we require the function type to unify with a function type where the argument type matches the argument:

$$\frac{\Gamma \vdash f : \tau_f \mid C_f \quad \Gamma \vdash x : \tau_x \mid C_x \quad \text{fresh } \beta}{\Gamma \vdash f x : \beta \mid C_f \cup C_x \cup \{(\tau_f, \tau_x \rightarrow \beta)\}} \quad (\text{APP})$$

3.3.5 Binary Operations

Binary operations require both operands to be integers:

$$\frac{\Gamma \vdash e_1 : \tau_1 \mid C_1 \quad \Gamma \vdash e_2 : \tau_2 \mid C_2}{\Gamma \vdash e_1 \text{ bop } e_2 : \text{int} \mid C_1 \cup C_2 \cup \{(\tau_1, \text{int}), (\tau_2, \text{int})\}} \quad (\text{BOP})$$

where $\text{bop} \in \{+, -, *, /, ==, <\}$.

3.3.6 Conditional

The condition must be an integer, and the two branches must have the same type:

$$\frac{\Gamma \vdash c : \tau_c \mid C_c \quad \Gamma \vdash t : \tau_t \mid C_t \quad \Gamma \vdash e : \tau_e \mid C_e}{\Gamma \vdash \text{if } c \text{ then } t \text{ else } e : \tau_t \mid C_c \cup C_t \cup C_e \cup \{(\tau_c, \mathbf{int}), (\tau_t, \tau_e)\}} \quad (\text{IF})$$

3.3.7 Vector Literal

All elements of a vector literal must have the same type:

$$\frac{\forall i : \Gamma \vdash e_i : \tau_i \mid C_i \quad \text{fresh } \alpha}{\Gamma \vdash \text{vec}[e_1, \dots, e_n] : \text{Vec}[\alpha] \mid \bigcup_i C_i \cup \bigcup_i \{(\tau_i, \alpha)\}} \quad (\text{VECLIT})$$

3.3.8 Vector Construction

`vecmk` takes a default value and a count. The count must be an integer:

$$\frac{\Gamma \vdash d : \tau_d \mid C_d \quad \Gamma \vdash c : \tau_c \mid C_c}{\Gamma \vdash \text{vecmk}[d, c] : \text{Vec}[\tau_d] \mid C_d \cup C_c \cup \{(\tau_c, \mathbf{int})\}} \quad (\text{VECMK})$$

3.3.9 Vector Length

`vecLen` takes a vector and returns an integer:

$$\frac{\Gamma \vdash v : \tau_v \mid C_v \quad \text{fresh } \alpha}{\Gamma \vdash \text{vecLen}[v] : \mathbf{int} \mid C_v \cup \{(\tau_v, \text{Vec}[\alpha])\}} \quad (\text{VECLLEN})$$

3.3.10 Vector Access

`vecget` takes a vector and an index (integer), returning an element:

$$\frac{\Gamma \vdash v : \tau_v \mid C_v \quad \Gamma \vdash i : \tau_i \mid C_i \quad \text{fresh } \alpha}{\Gamma \vdash \text{vecget}[v, i] : \alpha \mid C_v \cup C_i \cup \{(\tau_v, \text{Vec}[\alpha]), (\tau_i, \mathbf{int})\}} \quad (\text{VECGET})$$

3.3.11 Vector Update

`vecset` updates a vector at an index with a new value, returning the vector:

$$\frac{\Gamma \vdash v : \tau_v \mid C_v \quad \Gamma \vdash i : \tau_i \mid C_i \quad \Gamma \vdash \text{val} : \tau_{val} \mid C_{val} \quad \text{fresh } \alpha}{\Gamma \vdash \text{vecset}[v, i, \text{val}] : \text{Vec}[\alpha] \mid C_v \cup C_i \cup C_{val} \cup \{(\tau_v, \text{Vec}[\alpha]), (\tau_i, \mathbf{int}), (\tau_{val}, \alpha)\}} \quad (\text{VECSET})$$

3.3.12 Let-In Binding

Let-in bindings use polymorphic let, where the bound expression is type-checked and generalized before being used in the body:

$$\frac{\Gamma \vdash e : \tau_e \mid C_e \quad C_e \text{ is unified} \quad \sigma_e = \text{generalize}(\tau_e) \quad \Gamma, x : \sigma_e \vdash b : \tau_b \mid C_b}{\Gamma \vdash \text{let } x = e \text{ in } b : \tau_b \mid C_b} \quad (\text{LETIN})$$

3.4 Unification

The constraint set C is solved using the unification algorithm. Key unification rules:

$$\begin{aligned} \text{unify}(\mathbf{int}, \mathbf{int}) &= \emptyset \\ \text{unify}(\alpha, \alpha) &= \emptyset \\ \text{unify}(\text{Vec}[\tau_1], \text{Vec}[\tau_2]) &= \text{unify}(\tau_1, \tau_2) \\ \text{unify}(\tau_1 \rightarrow \tau_2, \tau_3 \rightarrow \tau_4) &= \text{unify}(\tau_1, \tau_3) \cup \text{unify}(\tau_2, \tau_4) \\ \text{unify}(\alpha, \tau) &= [\tau/\alpha] \quad \text{if } \alpha \notin \text{fv}(\tau) \text{ (occurs check)} \\ \text{unify}(\tau, \alpha) &= [\tau/\alpha] \quad \text{if } \alpha \notin \text{fv}(\tau) \text{ (occurs check)} \end{aligned}$$

fv stands for free type variables.

3.5 Generalization and Instantiation

After unifying all constraints in a let-binding block (with a single or multiple let-bindings), we generalize the resulting type(s) by universally quantifying over free type variables that are not in the outer scope:

$$\text{generalize}(\tau) = \forall \alpha_1 \dots \alpha_n. \tau$$

where $\alpha_1, \dots, \alpha_n$ are the free type variables in τ .

When using a polymorphic variable, we instantiate its scheme by replacing each quantified type variable with a fresh type variable:

$$\text{instantiate}(\forall \alpha_1 \dots \alpha_n. \tau) = \tau[\beta_1/\alpha_1, \dots, \beta_n/\alpha_n]$$

where $\beta_1, \dots, \beta_n$ are fresh type variables.

3.6 Strongly Connected Components (SCC) for Polymorphism

Since all let-bindings are mutually recursive, some (or even all) must be typechecked together. But typechecking all together by default can lead to less general types. Thus strongly connected components analysis determines which definitions can be typechecked independently. Definitions in the same SCC are treated as a single recursive group and generalized together.

4 Evaluation Strategy

intlang uses **eager evaluation** (also known as strict evaluation). All function arguments and sub-expressions are fully evaluated to values before being used, rather than being deferred as unevaluated computations.